Cyberscope

A TAC Security Company

Audit Report

EZManager

December 2025

Repository https://github.com/Tylergcantrell/EzManager

Commit c7f86daa7e02bea10476f90dc86b15cc9707da41

Audited by © cyberscope

Table of Contents

Table of Contents 1
Risk Classification 3
Review 4
Audit Updates 4
Source Files 4
Overview 5
CLCore 5
Valuation 6
Findings Breakdown 7
Diagnostics 8
IPD - Instantaneous Price Dependency 10
Description 10
Recommendation 11
CTV - Cross Tick Valuation 12
Description 12
Recommendation 13
SUV - Stale USDC Valuation 14
Description 14
Recommendation 15
SPSL - Suboptimal Pool Selection Logic 16
Description 16
Recommendation 16
UPSR - Uninitialized Pool Selection Risk 17
Description 17
Recommendation 17
AAO - Accumulated Amount Overflow 18
Description 18
Recommendation 18
AME - Address Manipulation Exploit 19
Description 19
Recommendation 20
CCR - Contract Centralization Risk 21
Description 21
Recommendation 22
MRP - Missing Reentrancy Protection 23
Description 23
Recommendation 23
PFRI - Potential Front Running Initialization 24
Description 24

Recommendation 25
PIDA - Potential Inconsistent Dust Adjustment 26
Description 26
Recommendation 26
PSU - Potential Subtraction Underflow 27
Description 27
Recommendation 27
SFO - Swap Fee Omission 28
Description 28
Recommendation 29
ZVE - Zero Value Estimation 30
Description 30
Recommendation 31
L04 - Conformance to Solidity Naming Conventions 32
Description 32
Recommendation 33
Functions Analysis 34
Inheritance Graph 38
Flow Graph 39
Summary 40
Disclaimer 41
About Cyberscope 42

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Repository	https://github.com/Tylergcantrell/EzManager
Commit	c7f86daa7e02bea10476f90dc86b15cc9707da41

Audit Updates

Initial Audit	23 Dec 2025

Source Files

Filename	SHA256
ICLDexAdapter.sol	43912944acc0f6cb39eb06012c2127ed0eb63deb8764c52d36d00e2a146472ab
CLCore.sol	a468122092d09b1211b69792de82ebe9e76f729a53ff2aefe5fd493a1a12f247
Valuation.sol	27ab09fb9f3fa77ddad4d438aefe4190e9515a6b426678c193358e667df27711

Overview

EZManager implements a Concentrated Liquidity Management System for managing, accounting, and valuing liquidity positions across Uniswap V3 and Aerodrome Slipstream. The architecture separates state/accounting (CLCore) from pricing logic (Valuation), providing modular tracking of Liquidity Provider (LP) positions and their USDC-equivalent values.

CLCore

CLCore serves as the central ledger of the system. It is an upgradeable, owner-controlled contract responsible for tracking positions, managing balances, and calculating accrued fees.

Key Responsibilities:

- Position Registry: Maintains a mapping of Position structs that link a unique key to: LP NFT (tokenId), Owner address, Tick range (tickLower, tickUpper), Deposited principal (totalDepositedUSDC), Pending fees and dust balances.
- Fee Accounting: Calculates accrued fees for positions by comparing the pool's pending fees against the position's snapshots. This provides consistent fee tracking across Uniswap V3 and Aerodrome pools.
- Dust Management: Tracks residual USDC that is not currently deployed in a liquidity range, ensuring precise accounting of user balances.
- Position Lifecycle Management: Supports registration, deregistration, and updating of positions, including adjustments to total deposited USDC.

Technical Highlights:

- Multi-DEX Support: Handles both standard Uniswap V3 and Aerodrome pools, including different tick spacing and fee structures.
- High-Precision Arithmetic: Uses TickMath, LiquidityAmounts, and FullMath for accurate calculations involving sqrt prices, liquidity, and fees.
- Upgradeable Architecture: Supports future enhancements via UUPS upgradeability.

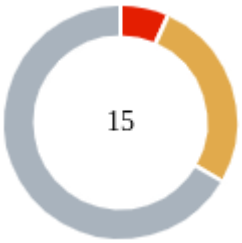
Valuation

Valuation is a view-only oracle contract that computes USDC-equivalent values for tokens held in the system.

Key Responsibilities:

- USDC Valuation: Converts token amounts into USDC-equivalent values.
- Routing Logic: Implements routing to identify the best pool based on liquidity. Supports direct USDC routing, and two-hop WETH paths.
- Multi-DEX Support: Handles both standard Uniswap V3 and Aerodrome pools, including different tick spacing and fee structures.
- Liquidity Thresholds: Ignores pools below a configurable minimum Liquidity threshold.
- TWAP Calculation: Computes a short (default 60-second) Time-Weighted Average Price. Falls back to instantaneous price if observations fail.

Findings Breakdown



Critical	1
Medium	4
Minor / Informative	10

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	1	0	0	0
Medium	4	0	0	0
Minor / Informative	10	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Critical	IPD	Instantaneous Price Dependency	Unresolved
Medium	CTV	Cross Tick Valuation	Unresolved
Medium	SUV	Stale USDC Valuation	Unresolved
Medium	SPSL	Suboptimal Pool Selection Logic	Unresolved
Medium	UPSR	Uninitialized Pool Selection Risk	Unresolved
Minor / Informative	AAO	Accumulated Amount Overflow	Unresolved
Minor / Informative	AME	Address Manipulation Exploit	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Unresolved
Minor / Informative	MRP	Missing Reentrancy Protection	Unresolved
Minor / Informative	PFRI	Potential Front Running Initialization	Unresolved
Minor / Informative	PIDA	Potential Inconsistent Dust Adjustment	Unresolved
Minor / Informative	PSU	Potential Subtraction Underflow	Unresolved
Minor / Informative	SFO	Swap Fee Omission	Unresolved

●	ZVE	Zero Value Estimation	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved

IPD - Instantaneous Price Dependency

Criticality	Critical
Location	core.sol valuation.sol
Status	Unresolved

Description

The valuation logic relies on the current state of the liquidity pool to determine asset prices, often using the instantaneous price rather than a time-weighted or averaged value. In certain edge cases, the contract defaults to this real-time price, making valuations sensitive to short-term market fluctuations.

This approach introduces a vulnerability to price manipulation, particularly through flash-loan attacks. An attacker can momentarily distort the pool's price, causing the contract to calculate an inflated or misleading USDC value for an asset. Since the valuation reflects the price at the exact moment of the call, even brief distortions can significantly impact the outcome.

```shell
function _spotQuote(
        address pool,
        address tokenIn,
        address tokenOut,
        uint256 amountIn,
        bool inIsT0
    ) internal view returns (uint256) {
    ...

}
```

Recommendation

The team should avoid relying solely on the instantaneous pool price. Instead, implement mechanisms that derive valuations from robust, time-averaged prices or aggregate data across multiple pools. This approach reduces susceptibility to short-term price manipulation and flash-loan attacks.

CTV - Cross Tick Valuation

Criticality	Medium
Location	valuation.sol
Status	Unresolved

Description

The contract estimates token valuations using the current pool liquidity and spot price but does not consider price shifts that occur when a swap crosses multiple ticks in Uniswap V3 pools. In Uniswap V3, liquidity is concentrated within specific tick ranges, and large trades can move the price across these ranges, altering the execution price. As a result, simply multiplying the current spot price by the swap amount may lead to inaccurate valuations. This approach can significantly overestimate or underestimate the actual USDC value for larger trades that impact multiple ticks.

```shell
function _spotQuote(
        address pool,
        address tokenIn,
        address tokenOut,
        uint256 amountIn,
        bool inIsT0
    ) internal view returns (uint256) {
    ...

}
```

Recommendation

Consider simulating swaps or incorporating tick-crossing logic to more accurately estimate the realized value for a given amount. This approach accounts for potential price impact, rather than relying solely on current liquidity and spot price. As a result, valuations become more precise and provide more reliable data for downstream logic.

SUV - Stale USDC Valuation

Criticality	Medium
Location	CLCore.sol#L504
Status	Unresolved

Description

The `updateTokenMetadata` function in CLCore updates a position's tokenId, tickLower, and tickUpper when a position is closed and reopened in a new range. However, it does not update the `totalDepositedUSDC` field, which represents an estimation of the gross USDC deposited into the position.

As a result, after a position is closed and reopened, the `totalDepositedUSDC` value remains stale and reflects the amount at the time of the previous opening. Any changes in USDC deposits or withdrawals that occur during the transition are not captured in the position metadata, leading to misrepresentation of the position's actual USDC value in on-chain queries or dependent calculations.

```shell
function updateTokenMetadata(bytes32 key, uint256 newTokenId,
int24 newTickLower, int24 newTickUpper)
external
onlyManager
whenNotPaused
{
if (newTokenId == 0) revert InvalidTokenId();

Position storage p = positions[key];
if (p.owner == address(0)) revert PositionNotFound();

uint256 oldId = p.tokenId;
int24 oldLower = p.tickLower;
int24 oldUpper = p.tickUpper;

p.tokenId = newTokenId;
p.tickLower = newTickLower;
```

```
p.tickUpper = newTickUpper;

emit PositionUpdated(key, oldId, newTokenId, oldLower,
oldUpper);

}
```

Recommendation

The team is advised to ensure that the `updateTokenMetadata` function also updates the `totalDepositedUSDC` value whenever a position is closed and reopened in a new tick range. This update should accurately reflect the amount of USDC deposited at the time of reopening, including any residual or leftover amounts.

SPSL - Suboptimal Pool Selection Logic

Criticality	Medium
Location	Valuation.sol
Status	Unresolved

Description

The Valuation contract currently selects the best pool for a given token based on liquidity (L) rather than the actual output amount in USDC. During both direct and two-hop Uniswap V3 or Aerodrome queries, a pool is chosen only if its liquidity exceeds the current maximum, even if another pool would yield a higher quote. Because Uniswap V3 liquidity is concentrated over specific price ranges, a highly liquid pool can offer a worse execution price than a pool with slightly lower liquidity. As a result, the contract may systematically produce inaccurate valuations that do not reflect the true market price.

```shell
if (ok && quote > 0 && L > localMaxLiquidity) {
    localMaxLiquidity = L;
    localBestQuote = quote;

}
```

Recommendation

The team is advised to modify the pool selection logic to prioritize the highest output amount (quote) instead of liquidity. The contract should select the pool providing the best execution price while still enforcing the minimum liquidity threshold, ensuring more accurate USDC valuations and reducing the risk of misrepresentation.

UPSR - Uninitialized Pool Selection Risk

Criticality	Medium
Location	Valuation.sol#L173,314
Status	Unresolved

Description

The contract's pool selection logic (IPSL), when combined with its dependence on instantaneous price data (IPD), introduces a potential vulnerability involving uninitialized pools. An attacker could initialize a previously unused pool and manipulate its liquidity and current price to make it appear as the most favorable option under the `localMaxLiquidity` comparison. Because the selection mechanism relies solely on liquidity metrics, a maliciously configured pool may be chosen even if it does not offer the best actual output. This can lead to inflated or misleading USDC valuations.

```shell
if (ok && quote > 0 && L > localMaxLiquidity) {
    localMaxLiquidity = L;
    localBestQuote = quote;

}
```

Recommendation

The team is advised to update the pool selection logic to prioritize the highest output amount (quote) rather than relying solely on liquidity. This approach ensures that even uninitialized or manipulated pools are only chosen if they offer the most accurate valuation, reducing the risk of inflated or misleading USDC valuations.

AAO - Accumulated Amount Overflow

Criticality	Minor / Informative
Location	CLCore.sol#L576
Status	Unresolved

Description

The contract is using variables to accumulate values. The contract could lead to an overflow when the total value of a variable exceeds the maximum value that can be stored in that variable's data type. This can happen when an accumulated value is updated repeatedly over time, and the value grows beyond the maximum value that can be represented by the data type.

```shell
function _nextKey() internal returns (bytes32) {
    unchecked { positionCounter++; }
    return keccak256(abi.encodePacked(address(this),
positionCounter, block.chainid));

}
```

Recommendation

The team is advised to carefully investigate the usage of the variables that accumulate value. A suggestion is to add checks to the code to ensure that the value of a variable does not exceed the maximum value that can be stored in its data type.

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	Valuation.sol#L142
Status	Unresolved

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. Within the scope of the valuation contract, both `dex` and `token` addresses are unsanitized. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

```shell
function usdcValue(address dex, address token, uint256 amount)
external view returns (uint256) {
if (amount == 0) return 0;
if (token == USDC) return amount;

address factory = ICLDexAdapter(dex).getFactory();
if (factory == address(0)) return 0;

bool isAerodrome = ICLDexAdapter(dex).isAerodrome();

(uint256 bestQuote, ) = isAerodrome
? _usdcValueAerodrome(factory, token, amount)
: _usdcValueUniswap(factory, token, amount);

return bestQuote;

}
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	CLCore.sol#L325,336,341,347,355,362,369,373 Valuation.sol#L120,129
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```shell
function _authorizeUpgrade(address) internal override
onlyOwner {}

function setMinLiquidity(uint128 _minLiquidity) external
onlyOwner {}
```

```shell
function setAllowedManager(address manager, bool allowed)
external onlyOwner whenNotPaused {}
function addBot(address bot) external onlyOwner whenNotPaused
{}
function removeBot(address bot) external onlyOwner
whenNotPaused {}
function setProtocolFeeBps(uint16 newBps) external onlyOwner
whenNotPaused {}
function setBotFeeBps(uint16 newBps) external onlyOwner
whenNotPaused {}
function pause() external onlyOwner {}
```

```
function unpause() external onlyOwner {}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

MRP - Missing Reentrancy Protection

Criticality	Minor / Informative
Location	CLCore.sol Valuation.sol
Status	Unresolved

Description

The contract executes complex DeFi operations based on user-provided input. However, it lacks adequate protection against advanced reentrancy exploits. Reentrancy can occur when a malicious actor is able to execute untrusted code during the execution of a function, or when internal calls to third functions are interleaved with the current execution flow. Such attacks can disrupt the normal logic of the contract, potentially leading to inconsistencies and loss of funds.

Recommendation

The team is advised to prevent potential reentrancy exploits, in line with Solidity best practices. To safeguard externally facing functions, the following measures can be considered:

- Add locks or mutexes within the function scope to prevent recursive calls.
- Ensure that external calls are made as the final operation in the function.
- Implement the "Checks-Effects-Interactions" pattern to maintain proper execution order and reduce vulnerability to reentrancy.

PFRI - Potential Front Running Initialization

Criticality	Minor / Informative
Location	CLCore.sol#L272 Valuation.sol#L102
Status	Unresolved

Description

The contract is vulnerable to a front-running risk on the `initialize` function, allowing an attacker to set arbitrary initial data. Since there are no access controls on `initialize`, any user can call this function upon deployment, regardless of intention. An attacker could front-run the initializer's transaction to claim ownership of the contract.

```shell
function initialize(address _USDC, address _WETH, address
owner_) external initializer {
if (_USDC == address(0) || _WETH == address(0) || owner_ ==
address(0)) revert ZeroAddress();

__Ownable_init(owner_);

USDC = _USDC;
WETH = _WETH;

// Default tier ordering
feeTiers[0] = 100;   // 0.01%
feeTiers[1] = 500;   // 0.05%
feeTiers[2] = 3000;  // 0.30%
feeTiers[3] = 10000; // 1.00%

// minLiquidity defaults to 0 (no filtering) and can be raised
by owner.
minLiquidity = 0;

}
```

Recommendation

It is recommended to implement measures to secure the initialization process. It is advised to implement robust safeguards and thorough documentation to mitigate the risks associated with external initialization.

PIDA - Potential Inconsistent Dust Adjustment

Criticality	Minor / Informative
Location	CLCore.sol#L529
Status	Unresolved

Description

The `adjustTotalDeposited` function in `CLCore` allows external calls from the manager to decrease a position's `totalDepositedUSDC` via a signed delta. If this adjustment is triggered based on dust calculations, the `totalDepositedUSDC` can be reduced inconsistently, as it did not account for any dust during its initiation.

```shell
function adjustTotalDeposited(bytes32 key, int256 usdcDelta)
external onlyManager whenNotPaused {
...
 p.totalDepositedUSDC -= dec;

 }
```

Recommendation

The team is advised to ensure that all calls to `adjustTotalDeposited` refer to the underlying amounts associated with the relevant position and accurately reflect the underlying assets. This helps maintain consistency in accounting and ensures that the contract's internal state aligns with the actual asset values.

PSU - Potential Subtraction Underflow

Criticality	Minor / Informative
Location	CLCore.sol#L795
Status	Unresolved

Description

The contract subtracts two values, the second value may be greater than the first value if the contract owner misuses the configuration. As a result, the subtraction may underflow and cause the execution to revert.

```shell
function _subUint256(uint256 a, uint256 b) internal pure
returns (uint256) {
unchecked {
    return a - b;
    }

}
```

Recommendation

The team is advised to properly handle the code to avoid underflow subtractions and ensure the reliability and safety of the contract. The contract should ensure that the first value is always greater than the second value. It should add a sanity check in the setters of the variable or not allow executing the corresponding section if the condition is violated. The team is advised to refrain from using unchecked operations.

SFO - Swap Fee Omission

Criticality	Minor / Informative
Location	Valuation.sol#L500
Status	Unresolved

Description

The contract's valuation logic omits swap fees when calculating output amounts. In the `_spotQuote` function, the quoted value is determined by multiplying the input amount by the current time-weighted average price (TWAP) or spot price, without incorporating the pool's fee tier. Consequently, the returned value represents a pre-fee estimate, whereas actual swaps on platforms like Uniswap V3 and Aerodrome always incur fees that reduce the final output.

This oversight can lead to systematic overvaluation, especially in high-fee pools or multi-hop swap paths where fees accumulate. As a result, any protocol logic that depends on these valuations may assume more favorable execution than is realistically achievable, leading to inaccurate pricing assumptions.

Shell
```
function _spotQuote(
    address pool,
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    bool inIsT0
) internal view returns (uint256) {
...

}
```

Recommendation

The team is advised to incorporate swap fee adjustments into the valuation logic by applying the pool's fee tier to the amount. The spot quote should reflect the execution value after fees, including compounded fees for multi-hop paths, to ensure valuations accurately represent the amount that would be received in a real swap.

ZVE - Zero Value Estimation

Criticality	Minor / Informative
Location	Valuation.sol
Status	Unresolved

Description

The valuation logic can return a zero USDC value in many instances instead of aborting the operation when a pool's price is unprocessable or exceeds the acceptable range. This behavior may be misleading, as it underreports or ignores valid asset values, potentially causing misrepresentation of protocol state or unintended financial impact.

```shell
if (sqrtPriceX96 > type(uint128).max) {
    return 0;

}
```

```shell
if (L < minLiquidity) {
    return (true, 0, 0);

}
```

```shell
if (inIsT0) {
if (tokenIn != t0 || tokenOut != t1) return 0;
} else {
if (tokenIn != t1 || tokenOut != t0) return 0;

}
```

Recommendation

The team is advised to handle out-of-range or unprocessable prices explicitly, rather than returning 0. This ensures that valuations more accurately reflect the true asset value and prevents silent misrepresentation.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	CLCore.sol#L127,128,129,323,808 Valuation.sol#L80,81,102,123,129
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```shell
address public USDC;
address public WETH;

/// Fee tiers to scan for Uniswap-style factories.
uint24[4] public feeTiers = [100, 500, 3000, 10000]; // 0.01%,
0.05%, 0.30%, 1.00%

/// Short TWAP window in seconds used for price estimation.
uint32 public constant TWAP_SECONDS = 60;

/// Q192 = 2^192, the denominator for Uniswap V3 priceX192.
uint256 private constant Q192 = 1 << 192;

/// Minimum liquidity threshold required for a pool/path to be
considered.
/// If no pools meet this threshold for a given token/dex,
valuation returns 0.

uint128 public minLiquidity;
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
CLCore	Implementation	Initializable, OwnableUpgradeable, IERC721Receiver, PausableUpgradeable, UUPSUpgradeable		
		Public	✓	-
	initialize	External	✓	initializer
	_authorizeUpgrade	Internal	✓	onlyOwner
	setAllowedManager	External	✓	onlyOwner whenNotPaused
	addBot	External	✓	onlyOwner whenNotPaused
	removeBot	External	✓	onlyOwner whenNotPaused
	setProtocolFeeBps	External	✓	onlyOwner whenNotPaused
	setBotFeeBps	External	✓	onlyOwner whenNotPaused
	pause	External	✓	onlyOwner
	unpause	External	✓	onlyOwner
	registerPosition	External	✓	onlyManager whenNotPaused
	deregisterPosition	External	✓	onlyManager whenNotPaused

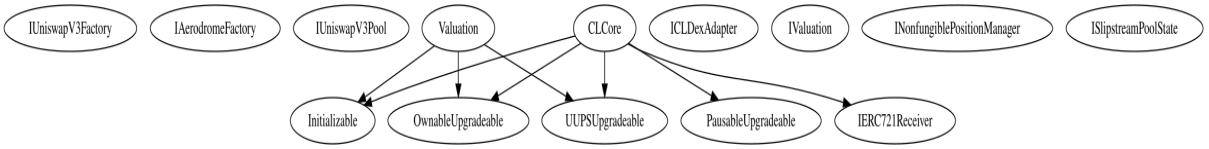
	addDustToPosition	External	✓	onlyManager whenNotPaused
	withdrawDustForPosition	External	✓	onlyManager whenNotPaused
	updateTokenMetadata	External	✓	onlyManager whenNotPaused
	adjustTotalDeposited	External	✓	onlyManager whenNotPaused
	setBotAllowedForPosition	External	✓	onlyManager whenNotPaused
	listUserPositionKeys	External		-
	_nextKey	Internal	✓	
	getPosition	External		-
	getPositionDetails	Public		-
	listUserPositionDetails	External		-
	positionValueUSDC	External		-
	pendingFees	External		-
	_pendingFeesForPosition	Internal		
	_positionValueUSDCSingle	Internal		
	_subUint256	Internal		
	_pendingFeesCalc	External		-
	onERC721Received	External		-
	_registerPosition	Internal	✓	
	_deregisterPosition	Internal	✓	
	_spotAmounts	Internal		
	_liquidityOf	Internal		

	_pool	Internal		
	_readSlot0	Internal		
	_valueTokenToUSDC	Internal		

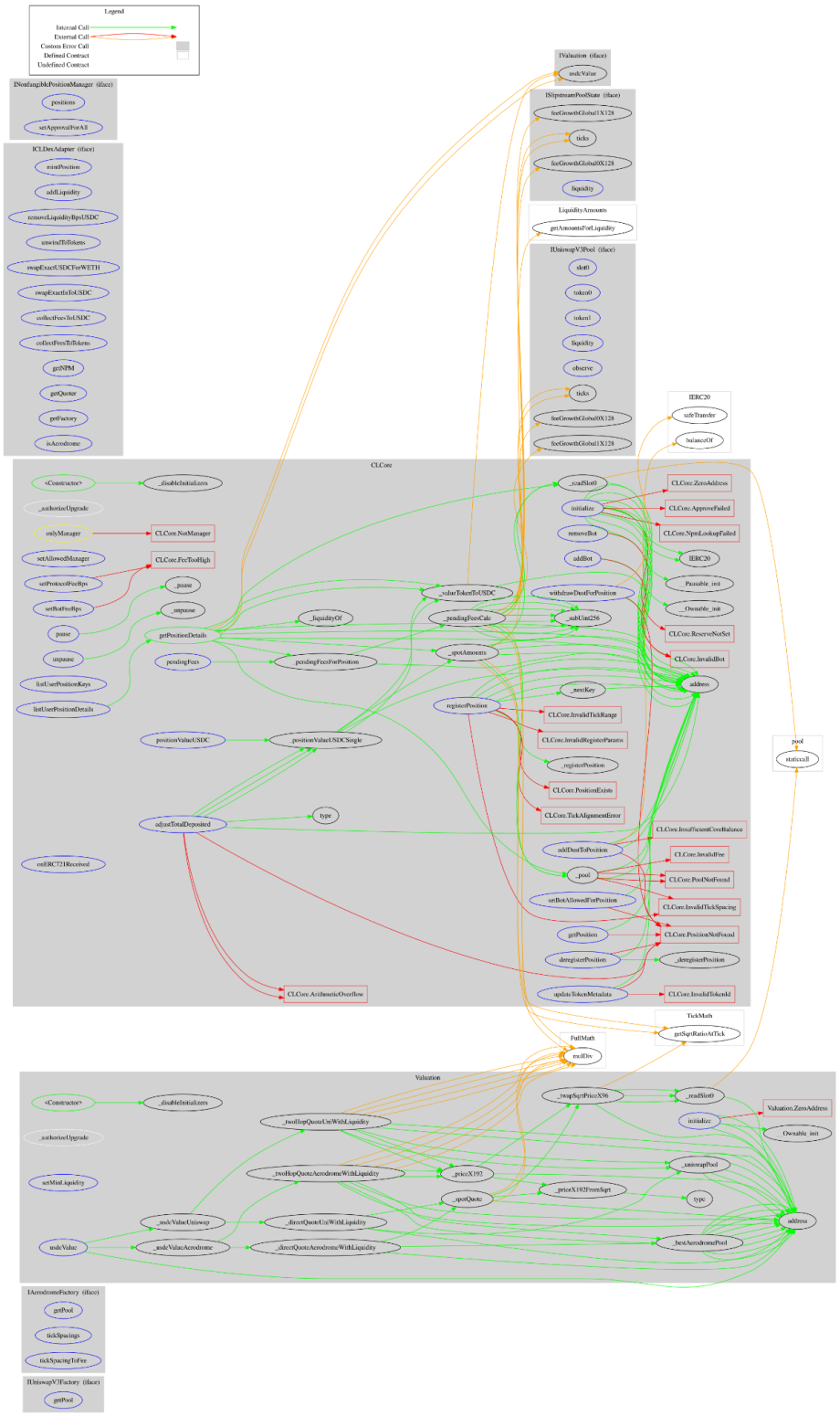
Valuation	Implementation	Initializable, OwnableUpgradeable, UUPSUpgradeable		
		Public	✓	-
	initialize	External	✓	initializer
	_authorizeUpgrade	Internal	✓	onlyOwner
	setMinLiquidity	External	✓	onlyOwner
	usdcValue	External		-
	_usdcValueUniswap	Internal		
	_directQuoteUniWithLiquidity	Internal		
	_twoHopQuoteUniWithLiquidity	Internal		
	_uniswapPool	Internal		
	_usdcValueAerodrome	Internal		
	_directQuoteAerodromeWithLiquidity	Internal		
	_twoHopQuoteAerodromeWithLiquidity	Internal		
	_bestAerodromePool	Internal		
	_priceX192	Internal		
	_priceX192FromSqrt	Internal		
	_twapSqrtPriceX96	Internal		
	_spotQuote	Internal		

	_readSlot0	Internal	

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
INonfungiblePositionManager (iface)
positions
setApprovalForAll
ICLDexAdapter (iface)
mintPosition
addLiquidity
removeLiquidityBpsUSDC
unwinfTfbTokens
swapExactUSDCForWETH
swapExactInToUSDC
collectFeesToUSDC
collectFeesToTokens
getNPM
getQuoter
getFactory
isAerodrome
IValuation (iface)
usdcValue
IUpstreamPoolState (iface)
feeGrowthGlobal1X128
ticks
feeGrowthGlobal0X124
liquidity
LiquidityAmounts
getAmountsForLiquidity
IUniswapV3Pool (iface)
slot0
token0
token1
liquidity
observe
ticks
feeGrowthGlobal0X124
feeGrowthGlobal1X124
IERC20
safeTransfer
balanceOf
CLCore
<Constructor>
_disableInitializers
_authorizeUpgrade
onlyManager
CLCore.NotManager
setAllowedManager
CLCore.FeeTooHigh
setProtocolFeeBps
setBotFeeBps
pause
unpause
listUserPositionKeys
listUserPositionDetails
_pause
_unpause
getPositionDetails
pendingFees
_liquidityOf
_pendingFeesForPosition
_valueTokenToUSDC
_pendingFeesCalc
_spotAmounts
withdrawDustForPosition
_subUint256
_readSlot0
initialize
removeBot
addBot
CLCore.ZeroAddress
CLCore.ApproveFailed
CLCore.NpmLookupFailed
IERC20
_Pausable_init
_Ownable_init
CLCore.ReserveNotSet
CLCore.InvalidBot
address
_nextKey
registerPosition
CLCore.InvalidTickRange
CLCore.InvalidRegisterParams
_registerPosition
CLCore.PositionExists
CLCore.TickAlignmentError
positionValueUSDC
_positionValueUSDCSingle
adjustTotalDeposited
type
onERC721Received
CLCore.ArithmeticOverflow
CLCore.InsufficientCoreBalance
addDustToPosition
CLCore.InvalidFee
_pool
CLCore.PoolNotFound
setBotAllowedForPosition
CLCore.InvalidTickSpacing
getPosition
CLCore.PositionNotFound
deregisterPosition
_deregisterPosition
updateTokenMetadata
CLCore.InvalidTokenId
pool
staticcall
TickMath
getSqrtRatioAtTick
FullMath
mulDiv
Valuation
<Constructor>
_disableInitializers
_authorizeUpgrade
setMinLiquidity
usdcValue
_usdcValueUniswap
_usdcValueAerodrome
_twoHopQuoteUniWithLiquidity
_twoHopQuoteAerodromeWithLiquidity
_directQuoteUniWithLiquidity
_directQuoteAerodromeWithLiquidity
_priceX192
_spotQuote
_twapSqrtPriceX96
_priceX192FromSqrt
_readSlot0
initialize
Valuation.ZeroAddress
_Ownable_init
_uniswapPool
type
address
_bestAerodromePool
IAerodromeFactory (iface)
getPool
tickSpacings
tickSpacingToFee
IUniswapV3Factory (iface)
getPool

Summary

The EZManager protocol proposes a liquidity management system, providing modular tracking, accounting, and valuation of LP positions across Uniswap V3 and Aerodrome Slipstream. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io